

Agents Lab

Samarth Bhatnagar

September 2026

Contents

Contents

1	Task 1: Understanding the Problem	2
2	Task 2: Designing the Agent	3
3	Task 3: Prompt Engineering	5
4	Final Code	6

1 Task 1: Understanding the Problem

1. What is the environment?

The environment is a warehouse represented as a two-dimensional grid. The grid contains free cells, represented by `.`, and obstacles (shelving units), represented by `#`. The starting position of the vehicle is represented by `S`, while the destination is represented by `G`. The vehicle can move between adjacent grid cells without crossing obstacles.

2. What is the goal of the agent?

The goal of the agent is to determine a collision-free path from the starting position `S` to the destination `G`. The agent must reach the goal while avoiding all obstacle cells in the warehouse.

3. What actions are available to the agent?

At each step, the vehicle can move one grid square in one of four directions:

- Up
- Down
- Left
- Right

A valid action must keep the vehicle within the warehouse and must not move it into an obstacle cell.

4. What information must the agent maintain in order to choose its next action?

The agent must maintain information about its current position in the warehouse, the location of the goal, and the locations of obstacles. During search, it must also keep track of the states that have already been explored and the paths or search information associated with them. This information allows the agent to determine which available action can move it toward the goal without colliding with an obstacle.

5. Why is this an example of a goal-based agent rather than a simple reflex agent?

This is a goal-based agent because its actions are selected with respect to an explicit goal: reaching the destination `G`. The agent must consider possible future states and search for a sequence of actions that leads from `S` to `G` while avoiding obstacles.

In contrast, a simple reflex agent selects an action directly from the current percept using predefined condition–action rules, without searching for a sequence of actions that achieves a future goal.

Think About It

If the warehouse becomes twice as large, the same general search strategy can still be used, provided that it is suitable for the larger state space. However, the search may have to explore many more states before finding the goal.

The additional difficulties include a larger search space, increased memory requirements for storing explored states and candidate paths, and potentially greater computation time. Therefore, the efficiency of the search algorithm becomes more important as the size of the warehouse increases.

2 Task 2: Designing the Agent

Before generating the program, the goal-based agent is designed by identifying its environment, current state, goal, available actions, and decision-making component.

1. Environment

The environment is the warehouse represented as a two-dimensional grid. It contains a starting position **S**, a goal position **G**, free cells represented by **.**, and obstacles represented by **#**. The agent must navigate through the free cells without entering obstacle cells.

2. Current State

The current state of the agent is its current position in the warehouse grid. It can be represented by the row and column coordinates of the vehicle:

$$s = (r, c)$$

where r is the row and c is the column of the vehicle's current position.

3. Goal

The goal is to reach the destination cell **G** starting from the cell **S**, while avoiding all obstacle cells.

Goal: reach G from S

4. Available Actions

The agent can move one grid square in any of the four directions:

- Up
- Down
- Left
- Right

An action is valid only if the resulting position is within the warehouse and is not occupied by an obstacle.

5. Decision-Making Component

The decision-making component is the search algorithm used by the agent to find a sequence of valid actions leading from the current state to the goal. It examines possible states and selects a path that reaches the destination without crossing obstacles.

Agent Architecture

The interaction between the components of the goal-based agent can be represented by the following block diagram.

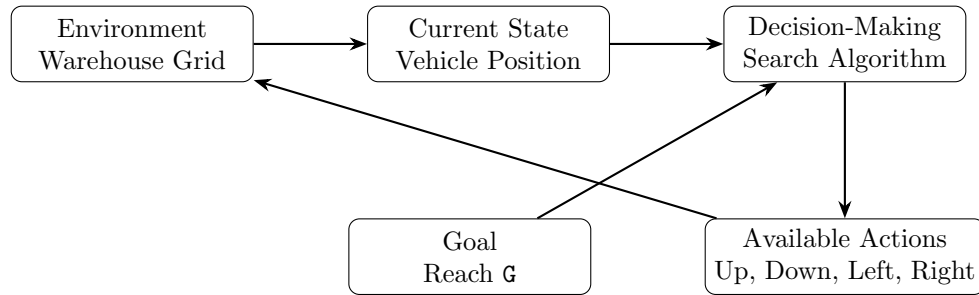


Figure 1: Design of the goal-based warehouse navigation agent.

The environment provides the current state of the vehicle and the warehouse layout. The decision-making component uses this information together with the goal to determine an appropriate action. The selected action changes the vehicle's state in the environment, and the process continues until the goal is reached.

3 Task 3: Prompt Engineering

Instead of asking the Large Language Model to simply “write a program”, a detailed specification was provided describing the environment, the required behaviour of the agent, and the expected output.

Prompt Used

The following prompt was given to the Large Language Model:

Write a well-documented Python program implementing a goal-based agent for the warehouse navigation problem shown in the laboratory sheet.

The program should:

- represent the warehouse as a two-dimensional grid;
- determine a collision-free path from **S** to **G**;
- avoid all obstacles;
- print either the path found or a suitable message if no path exists;
- explain the search algorithm that has been chosen and why it is appropriate.

The program should use the exact warehouse map given in the laboratory sheet and allow movement only in the four directions: Up, Down, Left, and Right.

After generating the program, test whether it runs correctly. Verify that the resulting path starts at **S**, ends at **G**, stays within the grid, and does not pass through any obstacle.

Execution and Testing

The generated Python program was executed using the warehouse map specified in the laboratory exercise. The program successfully found a collision-free path from **S** to **G**.

The program reported a path length of 20 moves. The path starts at (1,1), which is the location of **S**, and ends at (1,19), which is the location of **G**.

The resulting path was:

```
1 [(1, 1), (1, 2), (1, 3), (1, 4),  
2  (2, 4), (2, 5), (2, 6), (2, 7),  
3  (1, 7), (1, 8), (1, 9), (1, 10),  
4  (1, 11), (1, 12), (1, 13), (1, 14),  
5  (1, 15), (1, 16), (1, 17), (1, 18),  
6  (1, 19)]
```

Listing 1: Path generated by the warehouse navigation agent

The warehouse with the discovered path marked by * was:

```
1 #####
2 #S***.#####G#
3 #.#####. #
4 #...##.....#
5 #.#####.##.#.##. #
6 #.....#.....#
7 #####
```

Listing 2: Warehouse with the discovered path

The output confirms that the agent reaches the goal without entering any obstacle cell.

1. Did the LLM generate a working program on the first attempt?

Answer: Yes. The LLM generated a working program on the first attempt. After execution, the program successfully found a collision-free path of length 20 from S to G.

2. If not, how can the prompt be improved?

2. If not, how can the prompt be improved?

Answer: The question is not applicable because the generated program worked correctly on the first attempt. No correction to the prompt was required.

3. What search algorithm did the LLM choose?

Answer: The program uses Breadth-First Search (BFS). It maintains a queue of candidate paths and explores reachable positions level by level. A set of visited positions is used to avoid repeatedly exploring the same grid cell.

4. Why do you think the LLM selected this algorithm?

Answer: BFS is suitable for this warehouse because the vehicle moves between adjacent grid cells and each movement has the same cost. BFS systematically explores the reachable states and can find a shortest path in terms of the number of grid moves when a path exists. It is therefore a natural search strategy for this grid-based navigation problem.

4 Final Code

The following is the complete Python implementation generated with the assistance of the Large Language Model and used for the warehouse navigation experiment. The program represents the warehouse as a two-dimensional grid and uses Breadth-First Search (BFS) to find a collision-free path from S to G.

```

1  from collections import deque
2
3  # Warehouse map from the laboratory sheet
4  warehouse = [
5      "#####",
6      "#S...#.....G#",
7      "#.##...#####.#",
8      "#...#.....#.",
9      "#.#####.##.###.#",
10     "#.....#.....#",
11     "#####",
12 ]
13
14 # Movement directions:
15 # Up, Down, Left, Right
16 directions = [
17     (-1, 0), # Up
18     (1, 0),  # Down
19     (0, -1), # Left
20     (0, 1),  # Right
21 ]
22
23
24 def find_position(grid, symbol):
25     """Find the position of a symbol in the warehouse."""
26     for r, row in enumerate(grid):
27         for c, cell in enumerate(row):
28             if cell == symbol:
29                 return (r, c)
30     return None
31
32
33 def find_path(grid):
34     """
35     Find a collision-free path from S to G using BFS.
36     Returns the path as a list of positions.
37     """
38
39     start = find_position(grid, "S")
40     goal = find_position(grid, "G")
41
42     if start is None or goal is None:
43         return None
44
45     # Queue stores paths
46     queue = deque([[start]])
47
48     # Keep track of visited positions
49     visited = {start}
50
51     while queue:
52
53         path = queue.popleft()
54         current = path[-1]
55
56         # Goal reached
57         if current == goal:

```

```

58         return path
59
60     r, c = current
61
62     # Try all four possible actions
63     for dr, dc in directions:
64
65         new_r = r + dr
66         new_c = c + dc
67
68         # Check that the new position is inside the grid
69         if not (0 <= new_r < len(grid)):
70             continue
71
72         if not (0 <= new_c < len(grid[0])):
73             continue
74
75         # Check that the cell is not an obstacle
76         if grid[new_r][new_c] == "#":
77             continue
78
79         new_position = (new_r, new_c)
80
81         # Avoid visiting the same position repeatedly
82         if new_position in visited:
83             continue
84
85         visited.add(new_position)
86
87         # Add the new position to the path
88         new_path = path + [new_position]
89         queue.append(new_path)
90
91     # No path exists
92     return None
93
94
95 def print_path(grid, path):
96     """Print the warehouse with the discovered path."""
97
98     if path is None:
99         print("No path exists from S to G.")
100         return
101
102     path_set = set(path)
103
104     for r, row in enumerate(grid):
105
106         line = ""
107
108         for c, cell in enumerate(row):
109
110             if (r, c) in path_set and cell not in ("S", "G"):
111                 line += "*"
112             else:
113                 line += cell
114

```

```

115         print(line)
116
117
118     # Find the path
119     path = find_path(warehouse)
120
121     # Print the result
122     if path is not None:
123         print("Path found!")
124         print("Path length:", len(path) - 1)
125         print("Path:")
126         print(path)
127
128         print("\nWarehouse with path:")
129         print_path(warehouse, path)
130
131     else:
132         print("No path exists from S to G.")

```

Listing 3: Complete warehouse navigation agent